



**Министерство науки и высшего образования Российской
Федерации
Федеральное государственное бюджетное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА ИУ1 – СИСТЕМЫ АВТОМАТИЧЕСКОГО УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА № 3

«Синтез нелинейных систем»

по курсу:

«ОСНОВЫ ТЕОРИИ УПРАВЛЕНИЯ»

Студент: ИУ2-71

(Подпись, дата)

Аветисян Н. О.

Преподаватель:

(Подпись, дата)

Лобачев И.В.

2024 г.

Оглавление

Цель работы:	3
Задание:	3
Теоретическая часть	3
Моделирование нелинейной системы — математического сферического маятника	3
Метод компенсации нелинейностей	4
Моделирование нелинейной системы и синтез методом компенсации нелинейностей в среде MATLAB	5
Моделирование нелинейной системы	5
Синтез нелинейной системы методом компенсации нелинейностей	6
Общий вывод	9
Приложение	10
Код 1	10
Код 2	10
Код 3	12

Цель работы: исследование методов синтеза нелинейных систем с использованием метода компенсации нелинейностей.

Задание: Моделирование нелинейной системы — математического сферического маятника. Синтез нелинейной системы методом компенсации нелинейностей.

Теоретическая часть

Моделирование нелинейной системы — математического сферического маятника

Математический сферический маятник — это нелинейная динамическая система, которая для случая математического сферического маятника в сферической системе координат описывается следующей системой дифференциальных уравнений, приведенной к форме Коши:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = x_4^2 \cos x_1 \sin x_1 + \frac{mg}{l} \sin x_1 - \frac{\mu}{ml^2} x_2 + \frac{1}{ml^2} u_\theta \\ \dot{x}_3 = x_4 \\ \dot{x}_4 = -2x_2 x_4 \frac{\cos x_1}{\sin x_1} - \frac{\mu}{ml^2 \sin^2 x_1} x_4 + \frac{1}{ml^2 \sin^2 x_1} u_\varphi \end{cases} \quad (1)$$

Для данной системы вектор состояния задается в следующем виде:

$$\mathbf{x} = \begin{bmatrix} x_1 = \theta \\ x_2 = \dot{\theta} \\ x_3 = \varphi \\ x_4 = \dot{\varphi} \end{bmatrix}$$

Угол θ - угол, отсчитываемый от вертикальной оси.

Угол φ - угол, отсчитываемый от горизонтальной оси.

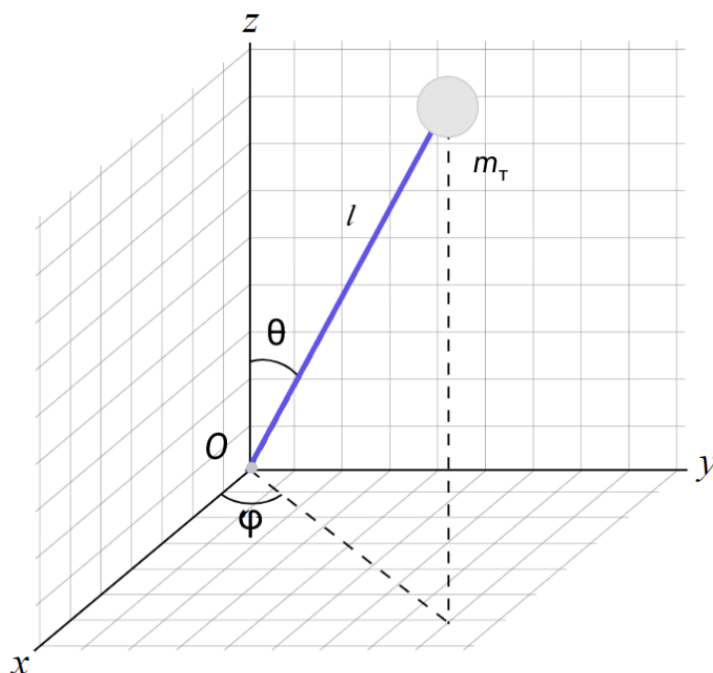


Рис. 1 Проекция маятника

В рассматриваемой системе управление по каждому углу $u_\theta(t)$ и $u_\varphi(t)$ формируется с целью регулирования – стабилизации системы в некотором заданном положении после перехода из некоторого исходного положения.

Для линейных систем синтез подобной системы регулирования достаточно легко решается с использованием различных вариантов ПИД-регуляторов. Для нелинейных систем сформировать ПИД-регулятор в явном виде не представляется возможным и для решения этой проблемы в данной работе предлагается использовать метод компенсации нелинейностей.

Метод компенсации нелинейностей

Метод компенсации нелинейностей применим для нелинейных систем, в которых закон управления может быть структурно выделен, т.е. уравнение состояния системы представимо в виде:

$$\dot{x}(t) = f(x, t) + g(x, t) u(t)$$

где $f(x, t)$, $g(x, t)$ в общем случае – нелинейные функции.

Тогда, сформировав закон регулирования $u(t)$ в следующем виде:

$$u(t) = \frac{-f(x,t) + v(t)}{g(x,t)}$$

получим линейную систему следующего вида:

$$\dot{x}(t) = v(t)$$

Для полученной линейной системы решается задача синтеза закона регулирования $v(t)$ любым известным и подходящим методом. Затем осуществляется возврат к исходной нелинейной системе, где $v(t)$ используется при вычислении $u(t)$ в соответствии с уравнением выше.

Моделирование нелинейной системы и синтез методом компенсации нелинейностей в среде MATLAB

Моделирование нелинейной системы

Напишем функцию `lab_ode()`, которая будет реализовывать вычисления правых частей системы дифференциальных уравнений записанных в форме Коши для рассматриваемой системы.

Код функции `lab_ode()` приведен в Приложении Код 1.

В коде основной программы задаем основные параметры математической системы и вектор начальных условий:

$$\begin{aligned} m &= 1 \text{ кг} \\ l &= 1 \text{ м} \\ \mu &= 0.5 \text{ Дж с}^2/\text{м}^2 \end{aligned} \quad \mathbf{x}_0 = \begin{bmatrix} \pi/2 \\ 0.1 \\ \pi/2 \\ 0.1 \end{bmatrix}$$

По исходным данным и результатам моделирования строим два графика.

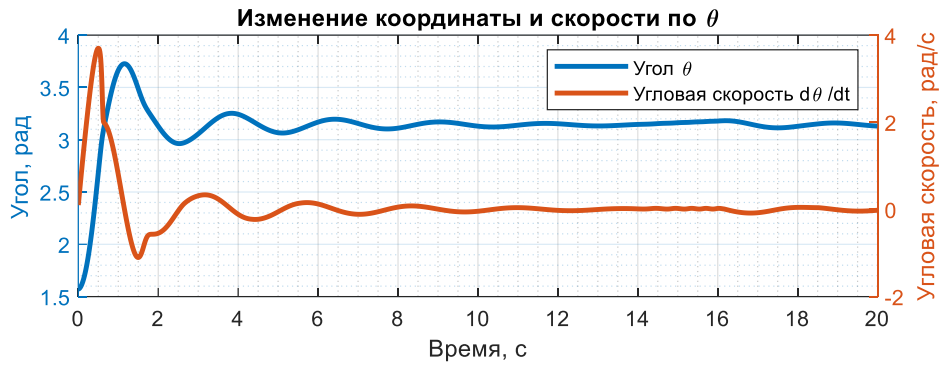


Рис. 2 Изменение скоростей и координат по θ

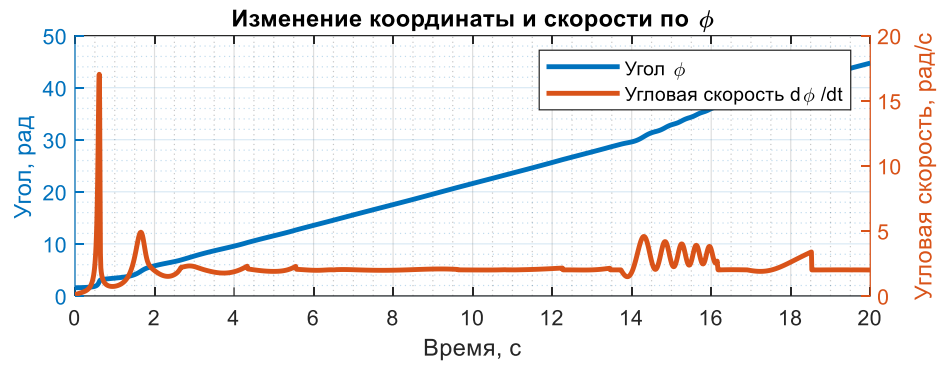


Рис. 3 Изменение скоростей и координат по ϕ

Синтез нелинейной системы методом компенсации нелинейностей

Для математической модели сферического маятника в форме Коши можно сформировать закон регулирования по каждому углу в следующем виде:

$$u_{\theta} = -ml^2 \left(x_4^2 \cos x_1 \sin x_1 + \frac{mg}{l} \sin x_1 \right) + ml^2 v_{\theta}$$

$$u_{\phi} = ml^2 \sin^2 x_1 \left(2x_2 x_4 \frac{\cos x_1}{\sin x_1} + \frac{\mu}{ml^2 \sin^2 x_1} x_4 - \frac{\mu}{ml^2} x_4 \right) + ml^2 \sin^2 x_1 v_{\phi}$$

Полученная после этого линейная система дифференциальных уравнений, приведенных к форме Коши примет вид:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = -\frac{\mu x_2}{ml^2} + v_{\theta} \\ \dot{x}_3 = x_4 \\ \dot{x}_4 = -\frac{\mu x_4}{ml^2} + v_{\phi} \end{cases}$$

Для полученных линейных систем задача регулирования решается с использованием ПИ-регуляторов по величинам ошибок углов отклонения (относительно заданных) и величинам ошибок угловых скоростей (относительно нулевых). В этом случае законы управления для описанных выше линейных систем можно записать в виде:

$$v_{\theta}(t) = \mu_{\theta}(t) + \mu_{\dot{\theta}}(t)$$

$$v_{\varphi}(t) = \mu_{\varphi}(t) + \mu_{\dot{\varphi}}(t)$$

где $\mu_{\theta}(t)$, $\mu_{\dot{\theta}}(t)$, $\mu_{\varphi}(t)$, $\mu_{\dot{\varphi}}(t)$ – соответствующие ПИ законы регулирования по углам и угловым скоростям. Ошибки по углу определяются следующим образом:

$$\theta_{\text{err}}(t) = \theta_{\text{зад}}(t) - \theta(t)$$

$$\varphi_{\text{err}}(t) = \varphi_{\text{зад}}(t) - \varphi(t)$$

а интегралы ошибок по углу вычисляются следующим:

$$\theta_{\text{err_int}}(t) = \theta_{\text{err_int}}(t) + \theta_{\text{err}}(t) \Delta t$$

$$\varphi_{\text{err_int}}(t) = \varphi_{\text{err_int}}(t) + \varphi_{\text{err}}(t) \Delta t$$

Аналогичным образом ПИ-законы регулирования по угловым скоростям примут вид:

$$\mu_{\dot{\theta}}(t) = k_{p\dot{\theta}} \dot{\theta}_{\text{err}}(t) + k_{i\dot{\theta}} \dot{\theta}_{\text{err_int}}(t)$$

$$\mu_{\dot{\varphi}}(t) = k_{p\dot{\varphi}} \dot{\varphi}_{\text{err}}(t) + k_{i\dot{\varphi}} \dot{\varphi}_{\text{err_int}}(t)$$

где $\dot{\theta}_{\text{err}}(t)$, $\dot{\varphi}_{\text{err}}(t)$, $\dot{\theta}_{\text{err_int}}(t)$, $\dot{\varphi}_{\text{err_int}}(t)$ – соответствующие ошибки по угловым скоростям. Отметим, что для определения коэффициентов ПИ-регуляторов можно использовать любой подходящий метод.

Вектор состояния баланс которого необходимо поддержать имеет следующий вид:

$$\mathbf{x}_k = \begin{bmatrix} 5/6 \pi \\ 0 \\ \pi/3 \\ 0 \end{bmatrix}$$

Начальные условия:

$$\mathbf{x}_0 = \begin{bmatrix} -\pi/2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Коэффициенты ПИ-регуляторов:

$$K_{p\theta} = K_{p\varphi} = 60.0$$

$$K_{i\theta} = K_{i\varphi} = 0.05$$

$$K_{p\dot{\theta}} = K_{p\dot{\varphi}} = 20.0$$

$$K_{i\dot{\theta}} = K_{i\dot{\varphi}} = 0.01$$

Создаем функцию `lab_ode_2()`, в которой реализуем моделирование нелинейной системы с синтезированным законом регулирования по методу компенсации нелинейностей. Код функции `lab_ode_2()` приведен в Приложении Код 2.

Тогда получаем:

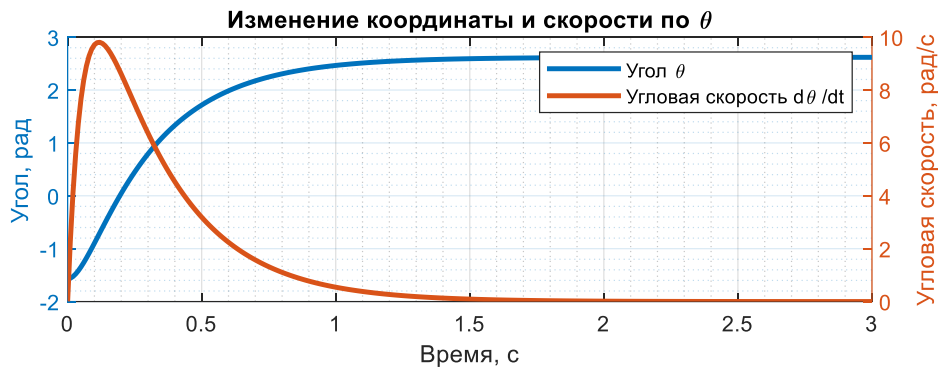


Рис. 4 Изменение скоростей и координат по θ

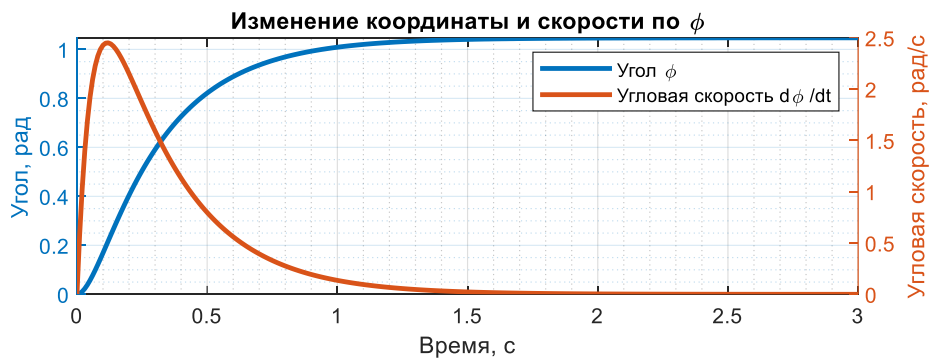


Рис. 5 Изменение скоростей и координат по φ

Ясно видно, что система регулирования успешно выполняет свою функцию: система переходит из исходного положения в заданное и затем стабилизируется.

Общий вывод

В данной работе исследована динамика нелинейной системы сферического маятника и проведён синтез управления с использованием метода компенсации нелинейностей. Управление реализовано с помощью ПИД-регуляторов для стабилизации углов θ и φ и их производных.

Моделирование включало изучение естественного поведения системы без управления и управление с компенсацией нелинейностей. Результаты численного моделирования подтвердили следующие выводы:

1. **Эффективность метода:** Метод компенсации нелинейностей позволил стабилизировать систему в окрестности целевых значений, несмотря на её нелинейный характер.
2. **Применение ПИД-регулятора:** ПИД-регуляторы обеспечили устойчивость и хорошие характеристики управления как в переходных, так и в установившихся режимах.

Таким образом, метод компенсации нелинейностей в сочетании с линейным управлением и ПИД-регуляторами доказал свою эффективность для управления нелинейными динамическими системами.

Приложение

Код 1

Функция lab_ode():

```
function dx = lab_ode(t, x)

    dx = zeros(4,1); % Инициализация вектора производных

    % Импорт глобальных переменных, необходимых для расчетов
    global m l mu u g

    % Уравнение движения для угла theta
    dx(1) = x(2);
    dx(2) = x(4)^2 * cos(x(1)) * sin(x(1)) ...
        + m * g / l * sin(x(1)) ...
        - mu / (m * l^2) * x(2) ...
        + 1 / (m * l^2) * u(1);

    % Уравнение движения для угла phi
    dx(3) = x(4);
    dx(4) = -2 * x(2) * x(4) * cos(x(1)) / sin(x(1)) ...
        - mu / (m * l^2 * (sin(x(1)))^2) * x(4) ...
        + 1 / (m * l^2 * (sin(x(1)))^2) * u(2);
end
```

Код 2

Функция lab_ode2():

```
function dx = lab_ode_2(t, x)

    dx = zeros(4,1);

    global m l mu g xk Kpt Kpdt Kit Kidt Kpp Kpdp Kip Kidp

    persistent t_prev theta_int phi_int dtheta_int dphi_int

    if (t == 0)

        t_prev = 0;
        theta_int = 0;
        phi_int = 0;
        dtheta_int = 0;
```

```

    dphi_int = 0;
end

% Ошибки по углу (от заданного значения)
err_theta = xk(1) - x(1);
err_phi = xk(3) - x(3);

% Ошибки по угловой скорости (от заданного значения)
err_dtheta = xk(2) - x(2);
err_dphi = xk(4) - x(4);

% Интегралы ошибок угловых положений
theta_int = theta_int + err_theta * (t - t_prev);
phi_int = phi_int + err_phi * (t - t_prev);

% Интегралы ошибок угловых скоростей
dtheta_int = dtheta_int + err_dtheta * (t - t_prev);
dphi_int = dphi_int + err_dphi * (t - t_prev);

% Регулирующие воздействия по углам
gamma_theta = Kpt * err_theta + Kit * theta_int;
gamma_phi = Kpp * err_phi + Kip * phi_int;

% Регулирующие воздействия по угловым скоростям
gamma_dtheta = Kpdt * err_dtheta + Kidt * dtheta_int;
gamma_dphi = Kpdp * err_dphi + Kidp * dphi_int;

v(1) = gamma_theta + gamma_dtheta;
v(2) = gamma_phi + gamma_dphi;

u(1) = -m*l^2*(x(4)^2*cos(x(1))*sin(x(1)) + m*g/l*sin(x(1))) + m*l^2*v(1);

u(2) = m*l^2*(sin(x(1)))^2*(2*x(2)*x(4)*cos(x(1))/sin(x(1)) + ...
    mu/(m*l^2*(sin(x(1)))^2)*x(4) - mu/(m*l^2)*x(4)) + ...
    m*l^2*(sin(x(1)))^2 * v(2);

dx(1) = x(2);
dx(2) = x(4)^2*cos(x(1))*sin(x(1)) + m * g/l *sin(x(1)) - mu/(m * l^2) *x(2) ...
    +1/(m*l^2) * u(1);
dx(3) = x(4);
dx(4) = -2*x(2)*x(4)*cos(x(1))/sin(x(1)) - mu/(m*l^2*(sin(x(1)))^2)*x(4) ...
    + 1/(m*l^2*(sin(x(1)))^2)*u(2);

t_prev = t;
end

```

Общий код:

```

clc
clear
close all

% Определение функций для решения ОДУ
ode_first = @lab_ode;
ode_second = @lab_ode_2;

% Глобальные переменные для параметров модели
global m l mu u g xk Kpt Kpdt Kit Kidt Kpp Kpdp Kip Kidp

% Инициализация параметров модели
m = 1;           % Масса
l = 1;           % Длина маятника
mu = 0.5;        % Коэффициент затухания
g = 9.815;       % Ускорение свободного падения

% Начальные условия для первой модели
X0 = [pi/2; 0.1; pi/2; 0.1];

% Управляющий вход для первой модели
u = [0; 1];

% Временной интервал моделирования
t = 0 : 0.01 : 20;

% Решение первой системы ОДУ
[~, X] = ode15s(ode_first, t, X0);

% Построение графиков для первой модели
figure
subplot(2,1,1)
yyaxis left
plot(t, X(:,1), 'LineWidth', 2)
ylabel('Угол, рад')
yyaxis right
plot(t, X(:,2), 'LineWidth', 2)
ylabel('Угловая скорость, рад/с')
title('Изменение координаты и скорости по \theta')
xlabel('Время, с')
legend('Угол \theta', 'Угловая скорость d\theta /dt')
grid on
grid minor

subplot(2,1,2)
yyaxis left
plot(t, X(:,3), 'LineWidth', 2)
ylabel('Угол, рад')
yyaxis right
plot(t, X(:,4), 'LineWidth', 2)
ylabel('Угловая скорость, рад/с')
title('Изменение координаты и скорости по \phi')
xlabel('Время, с')
legend('Угол \phi', 'Угловая скорость d\phi /dt')
grid on

```

```

grid minor

% Начальные условия для второй модели
X0 = [-pi/2; 0; 0; 0];

% Целевая точка для управления
xk = [5/6*pi; 0; pi/3; 0];

% Коэффициенты ПИД-регуляторов
Kpt = 60;
Kpdt = 20;
Kit = 0.05;
Kidt = 0.01;
Kpp = Kpt;
Kpdp = Kpdt;
Kip = Kit;
Kidp = Kidt;

% Временной интервал для второй модели
t = 0 : 0.01 : 3;

% Решение второй системы ОДУ
[~, X] = ode15s(ode_second, t, X0);

% Построение графиков для второй модели
figure
subplot(2,1,1)
yyaxis left
plot(t, X(:,1), 'LineWidth', 2)
ylabel('Угол, рад')
yyaxis right
plot(t, X(:,2), 'LineWidth', 2)
ylabel('Угловая скорость, рад/с')
title('Изменение координаты и скорости по \theta')
xlabel('Время, с')
legend('Угол \theta', 'Угловая скорость d\theta /dt')
grid on
grid minor

subplot(2,1,2)
yyaxis left
plot(t, X(:,3), 'LineWidth', 2)
ylabel('Угол, рад')
yyaxis right
plot(t, X(:,4), 'LineWidth', 2)
ylabel('Угловая скорость, рад/с')
title('Изменение координаты и скорости по \phi')
xlabel('Время, с')
legend('Угол \phi', 'Угловая скорость d\phi /dt')
grid on
grid minor

```